SERVICE BASED ARCHITECTURES

ACIT3855 – WINTER 2026

AGENDA

- Quick Review
- Quiz 1
- Microservices – Fowler
- Our Sample Application and First Service
  - Edge Service
  - Connexion
  - JSON, File I/O
  - Testing with PostMan and jMeter
- Lab 2
  - Edge Service

MARTIN FOWLER – SOFTWARE ARCHITECT AT THOUGHTWORKS

In short, the microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API. These services are **built around business capabilities** and **independently deployable** by fully automated deployment machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies.

-- James Lewis and Martin Fowler (2014)

MARTIN FOWLER – SOFTWARE ARCHITECT AT THOUGHTWORKS

Microservices provide benefits…

- Strong Module Boundaries: Microservices reinforce modular structure, which is particularly important for larger teams.
- Independent Deployment: Simple services are easier to deploy, and since they are autonomous, are less likely to cause system failures when they go wrong.
- Technology Diversity: With microservices you can mix multiple languages, development frameworks and data-storage technologies.

MARTIN FOWLER – SOFTWARE ARCHITECT AT THOUGHTWORKS

…but come with costs

- Distribution: Distributed systems are harder to program, since remote calls are slow and are always at risk of failure.
- Eventual Consistency: Maintaining strong consistency is extremely difficult for a distributed system, which means everyone has to manage eventual consistency.
- Operational Complexity: You need a mature operations team to manage lots of services, which are being redeployed regularly.

OPENAPI AND CONNEXION

- Let's review the sample OpenAPI Specification and Connexion Application from the reading

QUIZ 1

- Quiz is on the Learning Hub
- Open book
- You have ~10 minutes to complete it

COURSE SCHEDULE

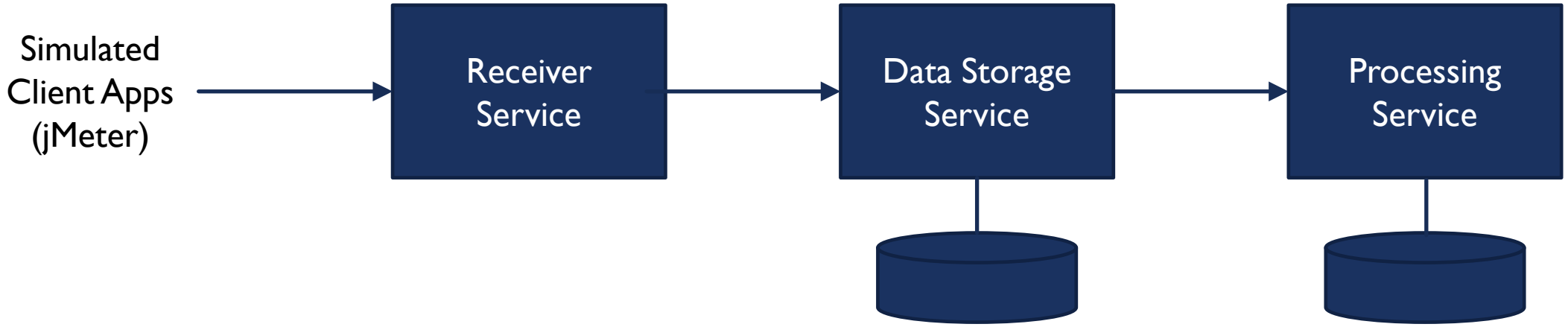
Week	Topics	Notes
1	• Services Based Architecture Overview • RESTful APIs Review	Lab 1
2	• Microservices Overview • Edge Service	Lab 2, Quiz 1
3	• Database Per Service • Storage Service (SQLite)	Lab 3, Quiz 2
4	• Logging, Debugging and Configuration • Storage Service (MySQL)	Lab 4, Quiz 3
5	• RESTful API Specification (OpenAPI) • Processing Service	Lab 5, Quiz 4
6	• Synchronous vs Asynchronous Communication • Message Broker Setup and Messaging	Lab 6, Quiz 5
7	• Event Sourcing • Analysis Service	Lab 7, Quiz 6
8	**Midterm Exam**	
9	• Deployment Part 1 - Containerization (Docker/Docker Compose)	Lab 8, Quiz 7
10	• Deployment Part 2 – Containerization (Docker Volumes and Bind Mounts)	Lab 9, Quiz 8
11	**Computing Reading Week**	
12	• Dashboard UI and CORS	Lab 10, Quiz 9
13	• Deployment Part 3 – Provisioning • Issues and Technical Debt	Lab 11, Quiz 10
14	• Deployment Part 4 – Load Balancing and Scaling • Final Exam Preview	Lab 12, Quiz 11
15	• Review Week	Lab 12 Demos, Assignment Due
16	**Final Exam**	

OUR SAMPLE APPLICATION

Our sample application will have three initial services:

- Receiver Service (Lab 1 and 2)
- Storage Service (Lab 3)
- Processing Service (Lab 5)

We will also be adding logging and external configuration to our services starting in Lab 4



EDGE SERVICE

- An **Edge Service** is one which is exposed to the public internet.
- Typically it receives requests from external applications, and routes them to the correct internal service within out application.
- An Edge Service could be implemented using a web application server, such as Nginx or Apache, acting as a reverse proxy or an API gateway
  - Reverse Proxy – We will set one of these up in ACIT 4850. A web server setup as a single endpoint for multiple web applications.
  - API Gateway – Specialized service that authenticates and routes incoming requests, and can limit incoming traffic. Usually provided by an open-source or commercial product.

We are going to build our own simple Edge Service (the Receiver Service) that receives our two Events.

CONNEXION

- Connexion – A Python Framework
  - Built on top of Flask
  - Automatically handles HTTP requests defined in an OpenAPI specification (i.e., 3.0.0)
  - https://connexion.readthedocs.io/en/latest/
- Your openapi.yml file defines your endpoints.
  - You add the openapi.yml file to your Connection application
  - You create a function for each endpoint with a name matching the operationId of the endpoint
  - Connexion automatically routes incoming requests to the correct function based on the operation id and passes in the request message as a parameter
  - JSON requests are automatically converted to Python objects (i.e., lists and/or dictionaries)

CONNEXION – UI DOCUMENTATION

localhost:8080/ui/#/

Reading API 1.0.0 OAS3

/openapi.json

This API receives reading events from medical devices

Contact the developer

Servers

devices Operations available to medical devices

POST /blood-pressure reports a blood pressure reading

POST /heart-rate reports a heart rate reading

REVIEW – PYTHON NAMING

- Remember our naming conventions in Python?
  - Functions and Variables?

    lower_snake_case

    Examples:
    first_name, x, systolic_bp
    get_response, report_bp_reading

  - Constant Values?

    UPPER_CASE

    Examples:
    PI
    NUM_READINGS

    These are typically defined at the top of our Python script or module.

REVIEW – FILE I/O AND OS.PATH.ISFILE

Reading from a file in Python

```
file_handle = open(filename,"r")
file_contents = file_handle.read()
file_handle.close()
```

Writing to a file in Python

```
file_handle = open(filename,"w")
file_handle.write(data_to_write)
file_handle.close()
```

os.path.isfile

- You will get an exception if you try to read from a file that doesn't exist
- import os.path
- os.path.isfile(filename) returns True if the file exists, False otherwise

REVIEW – FORMATTED STRING

- String Concatenation:
  - `desc = first + " " + last + " has a grade of " + str(grade) + "%"`
- Formatted String Expression:
  - `desc = "%s %s has a grade of %f%%" % (first, last, grade)`
- Format
  - `desc = "{} {} has a grade of {}%".format(first, last, grade)`
- F-strings
  - `desc = f"{first} {last} has a grade of {grade}%"`

REVIEW – DATETIME

```
import datetime

# Create a datetime object with the current date and time
current_datetime = datetime.datetime.now()

# Create a string with the datetime in the given format
current_datetime_str = current_datetime.strftime("%Y-%m-%d %H:%M:%S")
```

REVIEW – JSON MODULE

- Python has a built-in json module
  - https://docs.python.org/3/library/json.html
- Serialization – Convert Python data to a JSON string
  - json.dumps
  - json_str = json.dumps(python_data)
- Deserialization – Convert a JSON string to Python data
  - json.loads
  - python_data = json.loads(json_str)

You will use this in your Lab today to "log" requests to a file

TESTING – POSTMAN AND APACHE JMETER

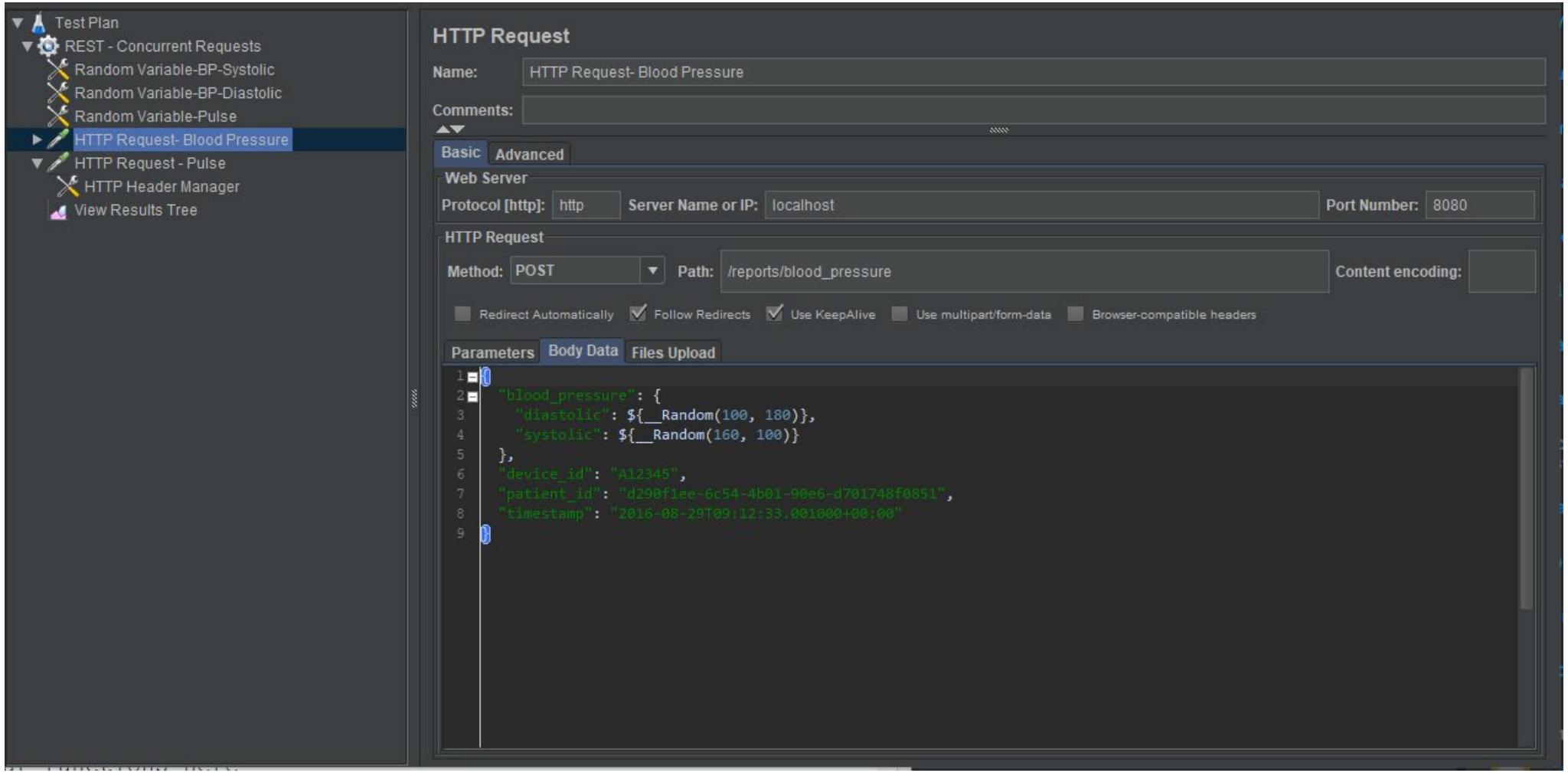
PostMan

- Should be familiar from ACIT 2515 and other classes
- Used to test RESTful API endpoints

Apache jMeter

- Java based tool that can be used to test the functionality and performance of web applications
- It allows us to create test scenarios as a series of HTTP requests
- We can have it apply through scenarios concurrently to simulate many users (i.e., high load on the system)

JMETER – EXAMPLE HTTP REQUEST



TODAY'S TOOLS

RESTful API Specification: SwaggerHub and OpenAPI

- Define a RESTful API in a yaml format (**Done in Lab 1**)

RESTful API Implementation: Python Connexion

- Built on top of Flask but allows integration with an OpenAPI specification

RESTful API Testing: PostMan and Apache jMeter

- Postman – same as ACIT 2515
- Apache jMeter – for load testing

You will be using these in your Lab today.

We will go through an example together in a moment.

DEMO – JMETER AND EDGE RECEIVER SERVICE

- Lets look at a sample stubbed out Edge Service using Connection
  - How to see the RESTful API documentation
  - File I/O
  - JSON
  - Algorithms – Fix Sized Queue
- Then we'll test it using PostMan and jMeter

TODAY'S LAB

You will be creating your Receiver Service (i.e., Edge Service) today in Lab 1 based on your OpenAPI specification from last week.

- It will receive each of your two batch events
- It will loop through the items in your batch to calculate a stat (i.e., average value)
- It will write data on those batch events as json data to one of two file (one file per event type)

You will be testing it out with PostMan (or equivalent) and Apache jMeter

Next week you will be creating a Storage Service and integrating it with your Receiver Service